IQGeo Senior SRE — Chaos Engineering & Observability

Gremlin • Kubernetes/EKS • Game Days • Proactive SRE • Prometheus • Grafana • OpenTelemetry

Interview objective: Be able to explain not only what each tool does, but how they work together to prove resilience in a production-like system. Keep spoken answers practical and evidence-driven.

1. The Core Architecture

Q: Explain how Gremlin, Kubernetes, Prometheus, Grafana and OpenTelemetry work together.

A: I see them as different layers of the same reliability workflow. Gremlin introduces a controlled failure into Kubernetes, such as pod termination, node failure or network degradation. Kubernetes should recover according to the desired state, while Prometheus measures availability, errors, latency and resource saturation. Grafana provides the operational view and lets me correlate the experiment with customer-facing SLOs. OpenTelemetry provides distributed traces so I can see exactly where a request was affected across services and dependencies. I then compare the observed behavior with the hypothesis, identify resilience gaps and rerun the experiment after remediation.

Q: What is the end-to-end flow?

A: The flow I keep in mind is: define hypothesis and steady state, inject a controlled fault, observe metrics/logs/traces, compare behavior against SLOs, mitigate if required, recover, perform RCA and turn the finding into engineering work. The key is that chaos is not random failure; it is controlled reliability validation.

2. Gremlin + Kubernetes

Q: How does Gremlin work with a Kubernetes cluster?

A: Gremlin integrates with Kubernetes so I can target specific workloads or infrastructure for controlled fault injection. Depending on the experiment, I can test pod termination, CPU or memory pressure, network degradation or node-level failure. I define the target, duration, blast radius, steady-state metrics and abort conditions before starting. Kubernetes should demonstrate its recovery mechanisms while Prometheus, Grafana and OpenTelemetry show the actual impact. The result is evidence about whether the platform is resilient, not simply whether a container restarted.

Q: What happens when Gremlin kills one pod?

A: Suppose a Deployment has five replicas and Gremlin terminates one pod. The Deployment/ReplicaSet controller detects that actual replicas are below the desired state and Kubernetes creates a replacement. The replacement becomes useful only after it passes readiness and is included in the Service endpoints. During that period I watch availability, error rate, latency, pod restart/startup time and traffic distribution. The experiment passes only if the customer-facing behavior remains within the predefined resilience boundary.

Q: Give a practical pod-kill experiment.

A: I would first establish baseline availability, error rate and latency. Then I would terminate one non-critical application pod with a small blast radius. I would verify that Kubernetes creates a replacement and that traffic is not sent to an unready pod. Prometheus would show the metric changes, Grafana would visualize the event, and OpenTelemetry would show whether individual requests experienced additional latency or failures. I would document the recovery time and rerun after any remediation.

Q: How would you test an EKS node failure?

A: I would first verify that workloads are distributed across multiple nodes and availability zones. Then I would target one worker node with a controlled failure and observe pod eviction/rescheduling, readiness, capacity and customer SLOs. Prometheus and Grafana would show node and workload behavior, while traces would help identify request-level latency during rescheduling. I would verify that the remaining capacity can absorb the workload and that recovery meets the expected objective.

Q: How would you test an Availability Zone failure?

A: I would treat this as a larger blast-radius experiment and only run it after validating smaller failure modes. The hypothesis would be that the application continues serving within its SLO using the remaining AZs. I would validate pod distribution, load-balancer behavior, application capacity, database availability and network routing. During the test I would monitor availability, latency, errors, saturation and traces. If the remaining AZs cannot absorb the load, that becomes a capacity or architecture gap to fix.

Q: How would you test network failure?

A: I would introduce a controlled latency, packet-loss or connectivity fault between selected components. For example, I could test the path between an EKS service and a dependency. I would watch timeout rates, retries, circuit breakers, request latency and dependency spans. Prometheus tells me how the service-level indicators change, while OpenTelemetry shows which downstream calls are affected. The goal is to verify graceful degradation rather than simply proving that the network can fail.

3. Chaos Engineering Fundamentals

Q: What is chaos engineering?

A: I see chaos engineering as deliberately introducing controlled failures to validate a system's resilience assumptions. I start with a hypothesis and define steady-state behavior and customer-facing SLOs. Then I inject the smallest useful fault, observe the system and stop or roll back if the defined safety boundary is exceeded. The important output is a reliability learning and a corrective action, not the failure itself.

Q: Chaos testing vs normal testing?

A: Normal testing validates expected behavior under known inputs and conditions. Chaos testing deliberately introduces failure conditions that may not occur during functional tests, such as pod loss, node failure, network latency or dependency failure. It validates redundancy, failover, retries, timeouts, alerting and recovery. I use it alongside functional, performance and security testing.

Q: What is a Game Day?

A: A Game Day is a planned resilience exercise where a team practices responding to a realistic failure scenario. It tests more than infrastructure: it can exercise monitoring, alerting, runbooks, escalation, communication and decision-making. I define the scenario, roles, success criteria and abort conditions beforehand. After the exercise I capture technical and process gaps and turn them into tracked improvements.

Q: What makes a chaos experiment safe?

A: I use a clearly defined hypothesis, small blast radius, approved target, time limit and explicit abort conditions. I avoid starting with the largest possible failure. The experiment needs reliable observability and a recovery path before execution. I also separate experiment control from normal production changes so the team can stop the test quickly if customer impact exceeds the agreed threshold.

Q: What if chaos testing reveals that the system does not recover?

A: I stop or roll back the experiment and restore customer service first. Then I identify the failed resilience assumption, such as insufficient replicas, bad readiness behavior, missing timeout controls, a single dependency or insufficient capacity. I document the finding, create a corrective action with an owner and rerun the same experiment after the fix. That gives us evidence that the improvement actually worked.

4. Microsoft-Style Game Day / Chaos Approach

Q: How does Microsoft use Game Days and chaos engineering?

A: Microsoft has publicly described chaos engineering and fault-injection practices as a controlled, hypothesis-driven way to improve resilience. Published Microsoft material describes Game Days where teams exercise realistic failure scenarios using normal monitoring and incident-management processes. Examples include failures involving compute, networking, databases and broader infrastructure. I would use that model as a reference for designing an IQGeo reliability exercise rather than claiming a specific internal Microsoft tool or process.

Q: Describe a Microsoft-style Game Day lifecycle.

A: I would structure it as hypothesis, preparation, controlled fault injection, observation, mitigation, recovery and review. Before the event, I define the expected steady state, customer SLOs, blast radius, monitoring, roles and abort criteria. During the event, the team uses normal operational signals and incident procedures. Afterward, the team reviews both technical recovery and operational response and creates corrective actions.

Q: What does a Game Day test besides technology?

A: It can test the people and process as well as the system. I would exercise on-call response, alert quality, escalation paths, runbooks, incident command, communication and recovery procedures. A system can technically fail over correctly but still have a poor operational outcome if nobody receives or understands the alert. That is why Game Days are useful for proactive SRE.

Q: How would you avoid overstating Microsoft experience?

A: I would say that Microsoft has publicly documented chaos engineering and Game Day practices, but I would not claim knowledge of confidential internal procedures or tools. I can use the published principles as an example of mature reliability engineering and explain how I would apply the same hypothesis-driven approach to the environment I am responsible for.

5. Prometheus During Chaos

Q: What does Prometheus contribute to chaos testing?

A: Prometheus provides the quantitative evidence. Before the experiment I establish a baseline for request rate, error rate, latency and saturation. During the experiment I watch those SLIs along with Kubernetes and infrastructure metrics. After recovery I compare the metrics with the baseline and expected SLO. That tells me whether the system actually remained reliable or only appeared to recover.

Q: How would you calculate API error rate?

A: I would calculate the rate of failed requests divided by the rate of total requests over a suitable time window. The exact metric names depend on the application instrumentation. I would visualize the result in Grafana and use an SLO-based alert rather than paging on every small transient increase.

Q: What is PromQL?

A: PromQL is Prometheus's query language. I use it to filter, aggregate and calculate rates or ratios from time-series metrics. For example, I can use rate over a counter to calculate requests per second and aggregate by service or status code. For repeated expensive queries, I can use recording rules.

Q: What are Prometheus labels?

A: Labels provide dimensions on time series, such as service, namespace, method or status code. They make aggregation and filtering powerful, but I need to control cardinality. I avoid putting unbounded values such as arbitrary user IDs into labels because that can create excessive time-series volume.

Q: What is high cardinality and why does it matter?

A: High cardinality means a metric has too many unique label combinations. It increases memory, storage and query cost and can degrade Prometheus performance. I keep labels to bounded dimensions that support real operational questions. For detailed per-request context, traces are generally a better place than high-cardinality metrics.

Q: What if Prometheus itself fails?

A: I treat observability as a production dependency and design it for resilience. Depending on the architecture, I would use redundant monitoring components, appropriate remote storage and independent alerting paths. Critical application health should not depend on one Prometheus instance. I would also monitor the observability platform itself with availability and ingestion health signals.

6. Grafana During Chaos and Incidents

Q: How would you design a chaos-testing Grafana dashboard?

A: I would start with customer SLIs: availability, request rate, error rate and latency percentiles. Then I would add pod restarts, readiness, node health, CPU, memory, network and dependency metrics. I would add deployment and chaos-experiment markers so the timeline is obvious. Finally I would provide links or panels for logs and traces so an engineer can move from symptom to root cause.

Q: What would an SRE dashboard show?

A: I would show SLO compliance and error-budget burn first, then golden signals and saturation. For Kubernetes I would add workload health, scheduling failures, resource pressure and restart rates. I would include deployment markers and dependency health. The dashboard should support a decision during an incident rather than simply contain many graphs.

Q: How does Grafana help correlate chaos with customer impact?

A: I can place the experiment start and end time alongside the service's SLI panels. If a pod is killed at 14:05 and error rate, P95 latency or dependency failures change immediately afterward, the relationship becomes visible. I can then use the trace and log links to investigate the exact request path. That gives the team a common operational view during the Game Day.

7. OpenTelemetry in Detail

Q: What is OpenTelemetry?

A: OpenTelemetry is a vendor-neutral observability framework for generating, collecting and exporting telemetry including traces, metrics and logs. I see it primarily as an instrumentation and telemetry pipeline rather than a visualization platform. It standardizes telemetry across services and allows the data to be sent to different backends.

Q: What is a trace?

A: A trace represents the end-to-end journey of one request through a distributed system. It contains related spans for operations such as an API handler, downstream HTTP call or database query. Trace context is propagated between services so those operations can be connected into one request path.

Q: What is a span?

A: A span represents one operation within a trace and typically contains timing, attributes, status and relationships to other spans. For example, an HTTP span can contain the route and status, while a child span can represent a PostgreSQL query. This helps me see exactly where time or errors are introduced.

Q: What is context propagation?

A: Context propagation carries identifiers such as trace context from one service to another. If Service A calls Service B, the trace context is propagated so the child operation remains associated with the same trace. Without propagation, distributed traces become fragmented and much less useful for incident diagnosis.

Q: What is the OpenTelemetry Collector?

A: The Collector is a vendor-neutral component that receives telemetry, processes it and exports it to observability backends. It can batch, filter, enrich, sample and route telemetry. This separates application instrumentation from a particular backend and gives the platform more control over the telemetry pipeline.

Q: How does OpenTelemetry help during a chaos experiment?

A: It provides request-level evidence. If a pod is killed and P95 latency increases, traces can show whether requests were delayed by pod startup, load balancing, a downstream dependency, retries or database activity. I correlate the trace timestamps with the chaos experiment and Prometheus metrics. That lets me move from a system-level symptom to the specific request path causing it.

Q: Prometheus vs OpenTelemetry?

A: Prometheus is primarily a metrics collection and time-series monitoring system. OpenTelemetry provides standardized instrumentation and collection for traces, metrics and logs and can export telemetry to different backends. They complement each other: Prometheus can provide strong metrics and SLO signals, while OpenTelemetry adds distributed telemetry and trace context.

8. Combined Incident Scenarios

Q: A pod is killed, it recovers, but latency spikes for 30 seconds. What do you investigate?

A: I would check readiness behavior, startup time, Service endpoint propagation and whether traffic was sent before the replacement was genuinely ready. Prometheus would show the latency and pod lifecycle changes, while OpenTelemetry traces could reveal whether the delay is application startup, network or a downstream dependency. I would also check cache or connection warm-up. The goal is to remove recovery-induced customer impact, not merely prove the pod restarted.

Q: Prometheus says everything is healthy but users report slow responses. What next?

A: I would not assume the system is healthy just because infrastructure metrics are green. I would check application-level latency and error metrics, then use OpenTelemetry traces to identify where request time is being spent. I would correlate that with logs, database latency and dependency behavior. This is a good example of why infrastructure monitoring alone is not sufficient observability.

Q: How do metrics, logs and traces complement each other?

A: Metrics tell me that a problem exists and quantify its scale. Logs provide detailed event and error context. Traces show the path of an individual request through distributed services and dependencies. I use all three together because a complex incident rarely has enough evidence in one signal alone.

Q: How would you prove a reliability improvement worked?

A: I define the expected improvement before making the change. Then I compare the relevant SLI, error-budget burn, incident rate, recovery time or failure behavior before and after. For resilience changes, I prefer rerunning the same chaos experiment and verifying that the system now meets the expected recovery objective. That turns the improvement into measurable evidence.

Q: How would you integrate chaos into CI/CD?

A: I would start with safe, repeatable experiments in lower environments after deployment and automated tests. A pipeline stage could deploy a candidate version, run a bounded chaos experiment, validate SLO-related metrics and stop promotion if the resilience criteria fail. More sensitive production experiments would be scheduled separately with approval and blast-radius controls. The principle is automated reliability validation without making uncontrolled production changes.

9. Rapid-Fire Client Questions

Q: What would you alert on: CPU or SLO?

A: I would generally prioritize customer-impacting SLO signals for paging. CPU is valuable as a diagnostic or capacity signal, but high CPU does not always mean customers are affected. I may alert on resource saturation when it is a reliable leading indicator of imminent customer impact.

Q: Why run chaos if Kubernetes already self-heals?

A: Self-healing is an assumption until it is validated under realistic failure. Kubernetes may recreate a pod, but the complete customer path could still experience latency, connection exhaustion or dependency failures. Chaos validates the whole system's recovery behavior, not just whether a container restarts.

Q: How do you reduce observability noise?

A: I remove redundant metrics and non-actionable alerts, standardize labels and control metric cardinality. I use sustained conditions, SLO-based alerting and appropriate trace sampling. The goal is that a page means a responder should take action, rather than creating alert fatigue.

Q: What if a chaos experiment causes a real outage?

A: I immediately follow the predefined abort and recovery procedure and focus on customer impact. I would stop the experiment, restore the service and communicate through the incident process. After recovery I would investigate why the safety boundary failed and improve the experiment controls before running it again.

Q: How would you mentor an engineer on chaos testing?

A: I would teach them to start with the hypothesis and steady state rather than jumping straight to a failure command. Then I would review the blast radius, observability, abort criteria and recovery plan with them. After the experiment I would ask them to explain the evidence and the reliability gap. The objective is to develop independent reliability judgment.

10. Commands and Practical Snippets

Q: Kubernetes incident commands

A: kubectl get pods -A
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get nodes
kubectl top nodes
kubectl top pods -A
kubectl get svc
kubectl get endpoints

Q: PromQL: request rate

A: rate(http_requests_total[5m])

Q: PromQL: approximate 5xx rate

A: sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))

Q: Linux quick health checks

A: uptime
free -h
df -h
top

11. The Best 60-Second Answer to Memorize

Q: Explain the full Gremlin + Kubernetes + Prometheus + Grafana + OpenTelemetry model.

A: I see them as a closed reliability feedback loop. Gremlin introduces a controlled failure into Kubernetes, such as pod, node or network failure. Kubernetes should recover according to its desired state, while Prometheus measures customer SLIs and infrastructure behavior. Grafana gives me the operational dashboard and lets me correlate the experiment with SLOs and error-budget impact. OpenTelemetry provides distributed traces so I can see exactly where individual requests were affected. I use that evidence to identify resilience gaps, remediate them and rerun the experiment to prove the system improved.

Final Mental Model

Chaos: Hypothesis → Steady state → Controlled fault → Observe → Abort/rollback → Recover → Analyze → Improve → Rerun.

Prometheus: Quantifies metrics and SLIs — traffic, errors, latency, saturation, Kubernetes health and capacity.

Grafana: Turns those signals into an operational view, SLO dashboards, experiment timelines and incident correlation.

OpenTelemetry: Adds distributed context — traces, spans and telemetry across services and dependencies.

Gremlin: Controlled fault injection — pod, node, resource and network failure experiments with bounded blast radius.

Game Day: Tests the technology, observability, runbooks, on-call response, escalation and recovery process together.

Senior SRE outcome: Don't stop at “the pod restarted.” Prove whether customer SLOs were protected, explain why recovery succeeded or failed, and show what engineering change prevents recurrence.